To limit the number of times we run the reorganization heuristic and avoid running it again on nodes that was recently re-clustered, we can introduce a threshold for the minimum number of points added/removed on the subtree rooted at each node, and only reorganize a node's children when this threshold is passed.

These methods proved to be effective in reducing the variance of the tree, producing more compact nodes.

But I'd like to conclude the discussion about these variants with a piece of advice: start implementing basic Ss-trees - at this point, you should be ready to implement your own version – profile them within your application (like we did for heaps in chapter 2), and only if they result critical, and improving their running time would reduce your application running time by at least 5-10%, try to implement one or more of the SS^+-tree heuristics presented in this section.

10.6 Summary

- To overcome the issues with k-d trees, alternative data structures like R-trees and Ss-trees have been introduced.
- The best data structure depends on the characteristics of the dataset and on how they need to be used, what's the dimension of the dataset, what's the distribution (shape) of the data, if the dataset is static or dynamic, if your application is search-intensive.
- Although R-trees and Ss-trees don't have any guarantee on the worst-case running time, in practice they